

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

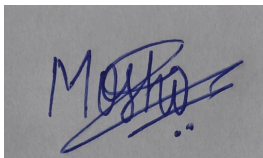
Coding Challenge

Code of Conduct:

I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink, appearing to read 'Moshika', is written over a grey rectangular background.

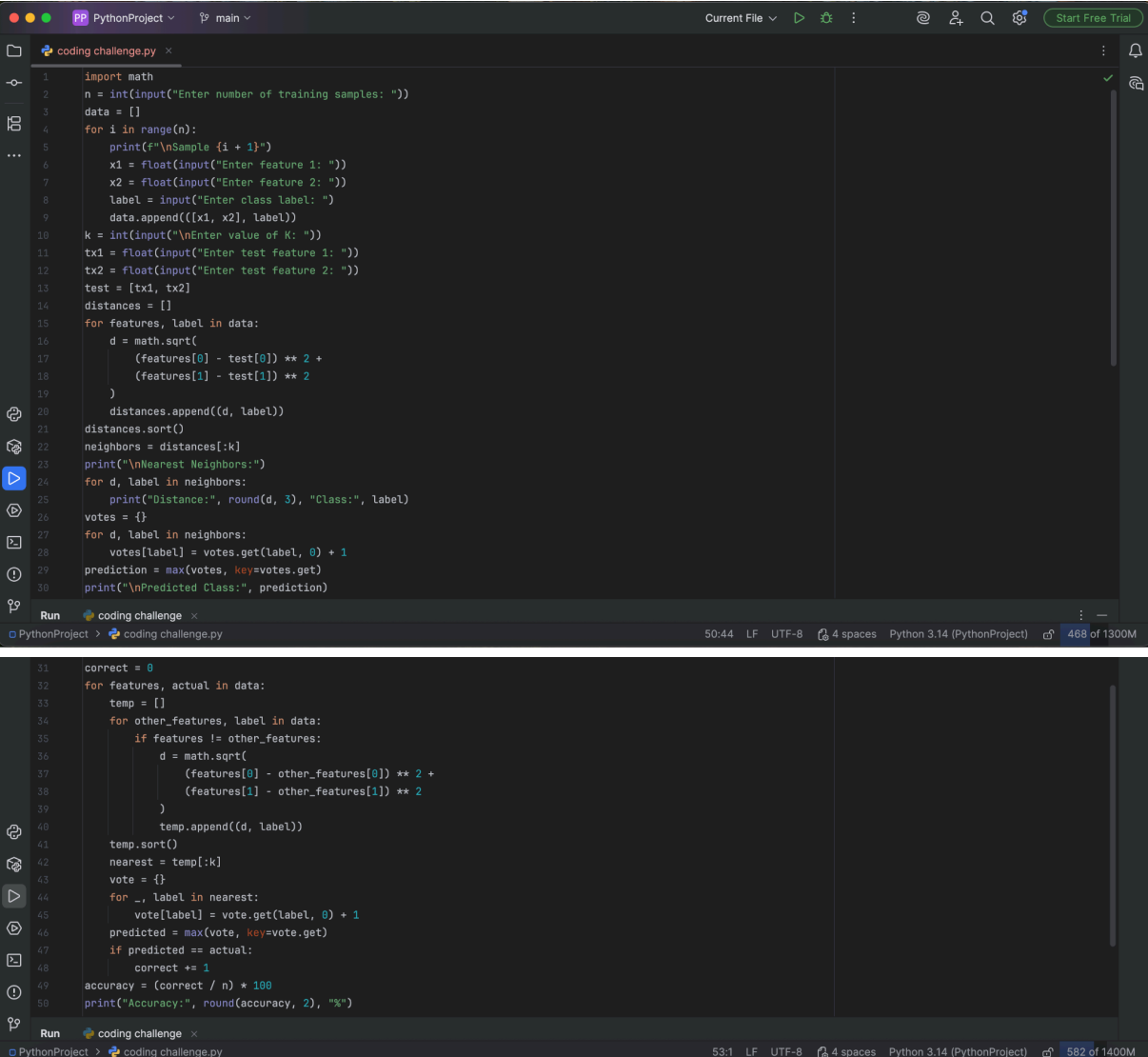
Annexure A

Short Answer Test

1. Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

ANSWER:

CODE



```
1 import math
2 n = int(input("Enter number of training samples: "))
3 data = []
4 for i in range(n):
5     print(f"\nSample {i + 1}")
6     x1 = float(input("Enter feature 1: "))
7     x2 = float(input("Enter feature 2: "))
8     label = input("Enter class label: ")
9     data.append([(x1, x2), label])
10 k = int(input("\nEnter value of K: "))
11 tx1 = float(input("Enter test feature 1: "))
12 tx2 = float(input("Enter test feature 2: "))
13 test = [tx1, tx2]
14 distances = []
15 for features, label in data:
16     d = math.sqrt(
17         (features[0] - test[0]) ** 2 +
18         (features[1] - test[1]) ** 2
19     )
20     distances.append((d, label))
21 distances.sort()
22 neighbors = distances[:k]
23 print("\nNearest Neighbors:")
24 for d, label in neighbors:
25     print("Distance:", round(d, 3), "Class:", label)
26 votes = {}
27 for d, label in neighbors:
28     votes[label] = votes.get(label, 0) + 1
29 prediction = max(votes, key=votes.get)
30 print("\nPredicted Class:", prediction)
31
32 correct = 0
33 for features, actual in data:
34     temp = []
35     for other_features, label in data:
36         if features != other_features:
37             d = math.sqrt(
38                 (features[0] - other_features[0]) ** 2 +
39                 (features[1] - other_features[1]) ** 2
40             )
41             temp.append((d, label))
42     temp.sort()
43     nearest = temp[:k]
44     vote = {}
45     for _, label in nearest:
46         vote[label] = vote.get(label, 0) + 1
47     predicted = max(vote, key=vote.get)
48     if predicted == actual:
49         correct += 1
50 accuracy = (correct / n) * 100
51 print("Accuracy:", round(accuracy, 2), "%")
```

OUTPUT

```
Run coding challenge x
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/coding_challenge.py
Enter number of training samples: 6

Sample 1
Enter feature 1: 1
Enter feature 2: 2
Enter class label: A

Sample 2
Enter feature 1: 2
Enter feature 2: 3
Enter class label: A

Sample 3
Enter feature 1: 3
Enter feature 2: 3
Enter class label: A

Sample 4
Enter feature 1: 7
Enter feature 2: 8
Enter class label: B

Sample 5
Enter feature 1: 8
Enter feature 2: 9
Enter class label: B

Sample 6
Enter feature 1: 9
Enter feature 2: 8
Enter class label: B

Enter value of K: 3
Enter test feature 1: 2.5
Enter test feature 2: 3

Nearest Neighbors:
Distance: 0.5 Class: A
Distance: 0.5 Class: A
Distance: 1.803 Class: A

Predicted Class: A
Accuracy: 100.0 %

Process finished with exit code 0
```

```
PythonProject > coding_challenge.py
53:1 LF UTF-8 4 spaces Python 3.14 (PythonProject) 527 of 1300M

Sample 6
Enter feature 1: 9
Enter feature 2: 8
Enter class label: B

Enter value of K: 3
Enter test feature 1: 2.5
Enter test feature 2: 3

Nearest Neighbors:
Distance: 0.5 Class: A
Distance: 0.5 Class: A
Distance: 1.803 Class: A

Predicted Class: A
Accuracy: 100.0 %

Process finished with exit code 0

PythonProject > coding_challenge.py
53:1 LF UTF-8 4 spaces Python 3.14 (PythonProject) 571 of 1300M
```

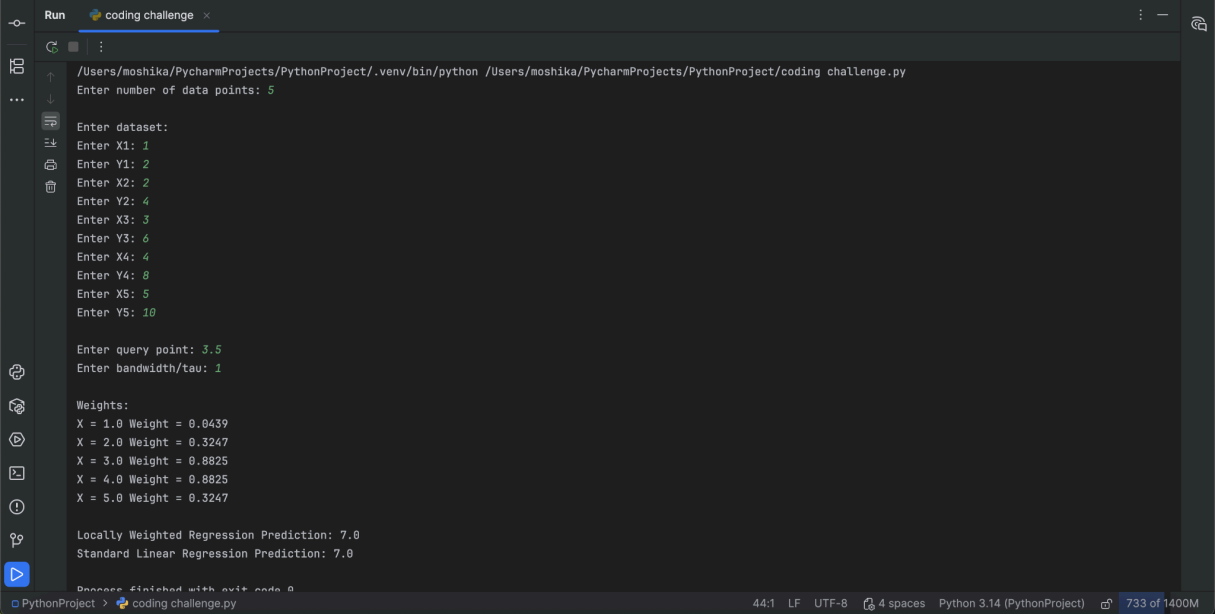
2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

ANSWER:

CODE

```
1 import numpy as np
2 n = int(input("Enter number of data points: "))
3 X = []
4 Y = []
5 print("\nEnter dataset:")
6 for i in range(n):
7     x = float(input(f"Enter X{i + 1}: "))
8     y = float(input(f"Enter Y{i + 1}: "))
9     X.append(x)
10    Y.append(y)
11 X = np.array(X)
12 Y = np.array(Y)
13 query = float(input("\nEnter query point: "))
14 tau = float(input("Enter bandwidth/tau: "))
15 weights = np.exp(
16     -((X - query) ** 2) /
17     (2 * tau ** 2)
18 )
19 X_matrix = np.column_stack(
20     (np.ones(len(X)), X)
21 )
22 W = np.diag(weights)
23 theta = np.linalg.pinv(
24     X_matrix.T @ W @ X_matrix
25 ) @ X_matrix.T @ W @ Y
26 lwr_prediction = theta[0] + theta[1] * query
27 coefficients = np.polyfit(X, Y, deg: 1)
28 linear_prediction = (
29     coefficients[0] * query +
30     coefficients[1]
31 )
32 print("\nWeights:")
33 for i in range(n):
34     print(
35         "X =", X[i],
36         "Weight =", round(weights[i], 4)
37     )
38 print("\nLocally Weighted Regression Prediction:",
39     round(lwr_prediction, 4))
40 print("Standard Linear Regression Prediction:",
41     round(linear_prediction, 4))
```

OUTPUT



The screenshot shows a Jupyter Notebook terminal window titled "coding challenge". The terminal output is as follows:

```
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/coding_challenge.py
Enter number of data points: 5

Enter dataset:
Enter X1: 1
Enter Y1: 2
Enter X2: 2
Enter Y2: 4
Enter X3: 3
Enter Y3: 6
Enter X4: 4
Enter Y4: 8
Enter X5: 5
Enter Y5: 10

Enter query point: 3.5
Enter bandwidth/tau: 1

Weights:
X = 1.0 Weight = 0.0439
X = 2.0 Weight = 0.3247
X = 3.0 Weight = 0.8825
X = 4.0 Weight = 0.8825
X = 5.0 Weight = 0.3247

Locally Weighted Regression Prediction: 7.0
Standard Linear Regression Prediction: 7.0

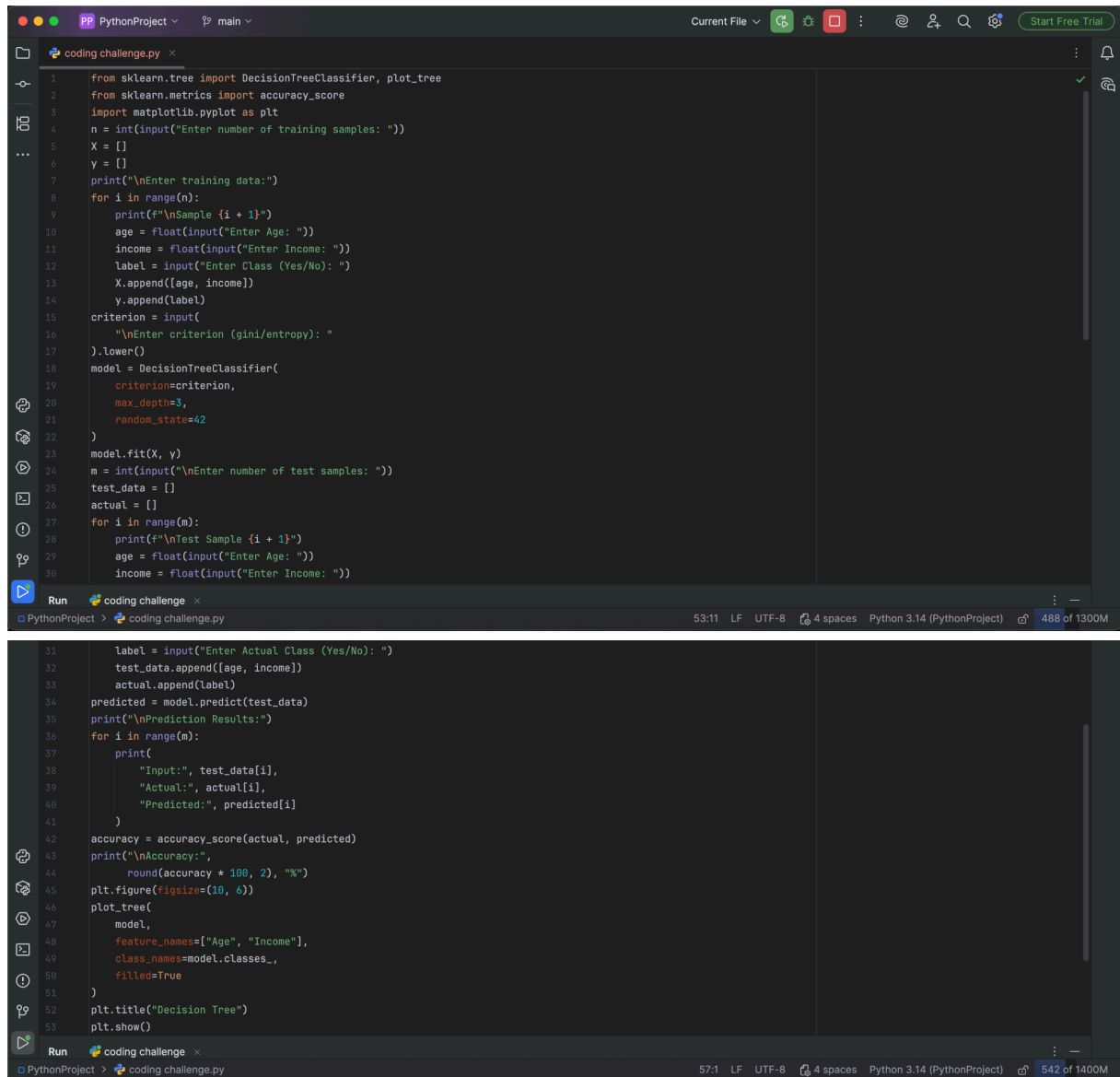
Process finished with exit code 0
```

The bottom status bar of the terminal window displays the following information: 44:1 LF UTF-8 4 spaces Python 3.14 (PythonProject) 733 of 1400M.

6. Write a Python program to implement the Decision Tree algorithm for classification. Use a small dataset and construct the tree using information gain or Gini index. Allow prediction for test instances and display accuracy. Visualize the tree structure and analyze how feature selection impacts classification performance.

ANSWER:

CODE



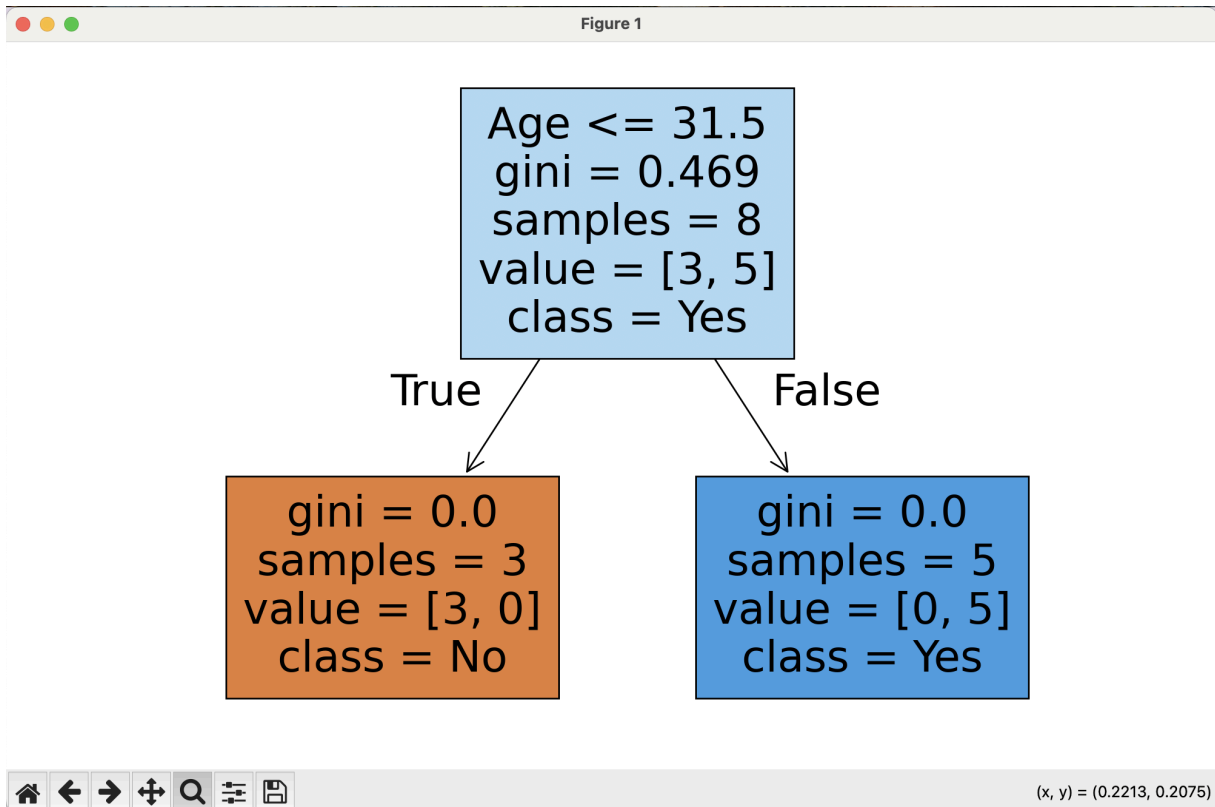
```
1 from sklearn.tree import DecisionTreeClassifier, plot_tree
2 from sklearn.metrics import accuracy_score
3 import matplotlib.pyplot as plt
4 n = int(input("Enter number of training samples: "))
5 X = []
6 y = []
7 print("\nEnter training data:")
8 for i in range(n):
9     print(f"Sample {i + 1}:")
10    age = float(input("Enter Age: "))
11    income = float(input("Enter Income: "))
12    label = input("Enter Class (Yes/No): ")
13    X.append([age, income])
14    y.append(label)
15    criterion = input(
16        "\nEnter criterion (gini/entropy): "
17    ).lower()
18    model = DecisionTreeClassifier(
19        criterion=criterion,
20        max_depth=3,
21        random_state=42
22    )
23    model.fit(X, y)
24    m = int(input("\nEnter number of test samples: "))
25    test_data = []
26    actual = []
27    for i in range(m):
28        print(f"Test Sample {i + 1}:")
29        age = float(input("Enter Age: "))
30        income = float(input("Enter Income: "))
31        label = input("Enter Actual Class (Yes/No): ")
32        test_data.append([age, income])
33        actual.append(label)
34    predicted = model.predict(test_data)
35    print("\nPrediction Results:")
36    for i in range(m):
37        print(
38            "Input:", test_data[i],
39            "Actual:", actual[i],
40            "Predicted:", predicted[i]
41        )
42    accuracy = accuracy_score(actual, predicted)
43    print("\nAccuracy:",
44        round(accuracy * 100, 2), "%")
45    plt.figure(figsize=(10, 6))
46    plot_tree(
47        model,
48        feature_names=["Age", "Income"],
49        class_names=model.classes_,
50        filled=True
51    )
52    plt.title("Decision Tree")
53    plt.show()
```

OUTPUT

```
Run coding challenge x
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/coding_challenge.py
Enter number of training samples: 8
Enter training data:
Sample 1
Enter Age: 22
Enter Income: 25000
Enter Class (Yes/No): No
Sample 2
Enter Age: 25
Enter Income: 30000
Enter Class (Yes/No): No
Sample 3
Enter Age: 28
Enter Income: 35000
Enter Class (Yes/No): No
Sample 4
Enter Age: 35
Enter Income: 50000
Enter Class (Yes/No): Yes
```

```
Run coding challenge x
Sample 5
Enter Age: 40
Enter Income: 60000
Enter Class (Yes/No): Yes
Sample 6
Enter Age: 45
Enter Income: 70000
Enter Class (Yes/No): Yes
Sample 7
Enter Age: 50
Enter Income: 80000
Enter Class (Yes/No): Yes
Sample 8
Enter Age: 55
Enter Income: 90000
Enter Class (Yes/No): Yes
Enter criterion (gini/entropy): gini
Enter number of test samples: 3
Test Sample 1
Enter Age: 24
Enter Income: 20000
Enter Actual Class (Yes/No): No
```

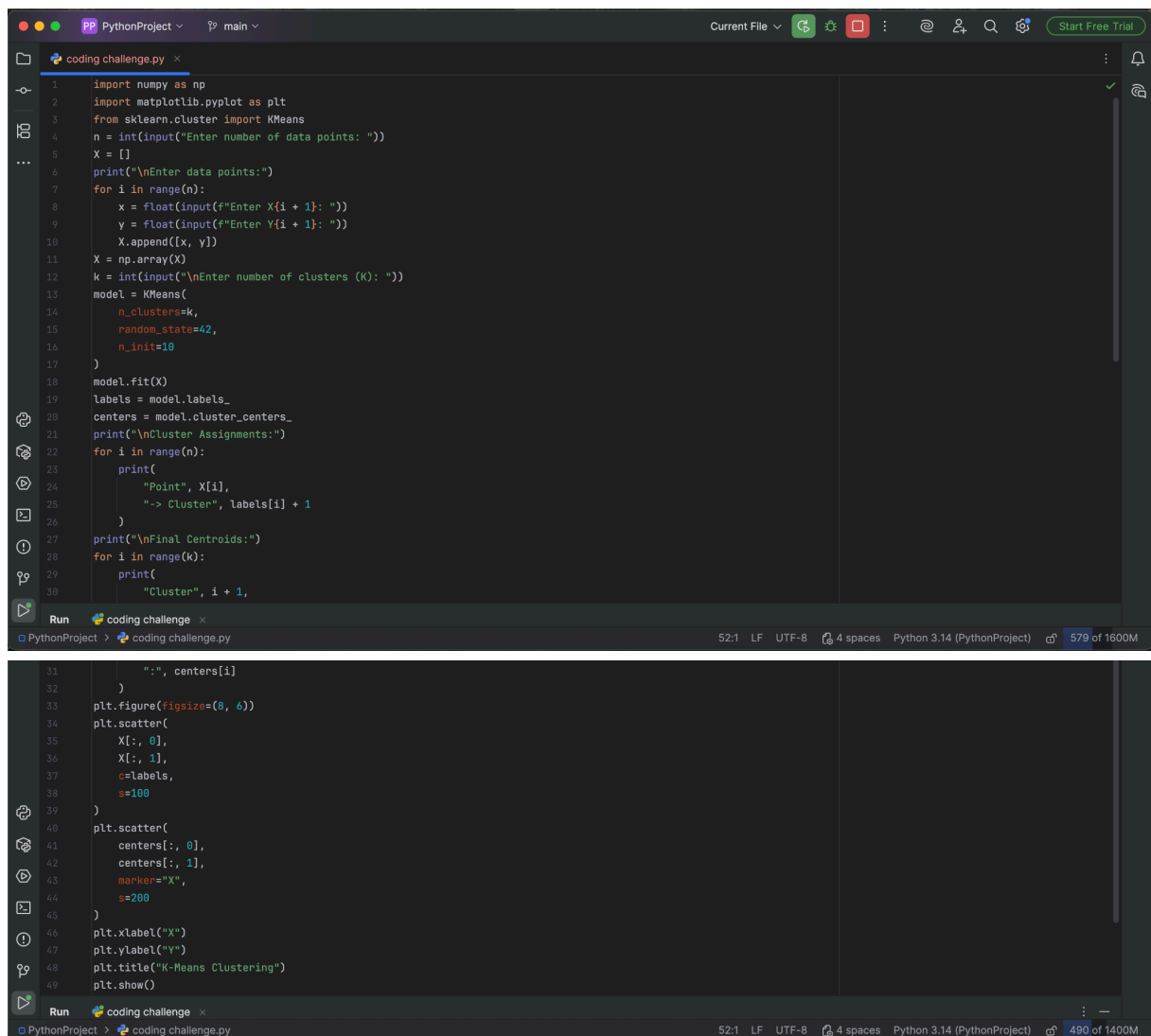
```
Test Sample 2
Enter Age: 42
Enter Income: 65000
Enter Actual Class (Yes/No): Yes
Test Sample 3
Enter Age: 52
Enter Income: 85000
Enter Actual Class (Yes/No): Yes
Prediction Results:
Input: [24.0, 20000.0] Actual: No Predicted: No
Input: [42.0, 65000.0] Actual: Yes Predicted: Yes
Input: [52.0, 85000.0] Actual: Yes Predicted: Yes
Accuracy: 100.0 %
```



9. Create a Python program to implement the K-Means clustering algorithm. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

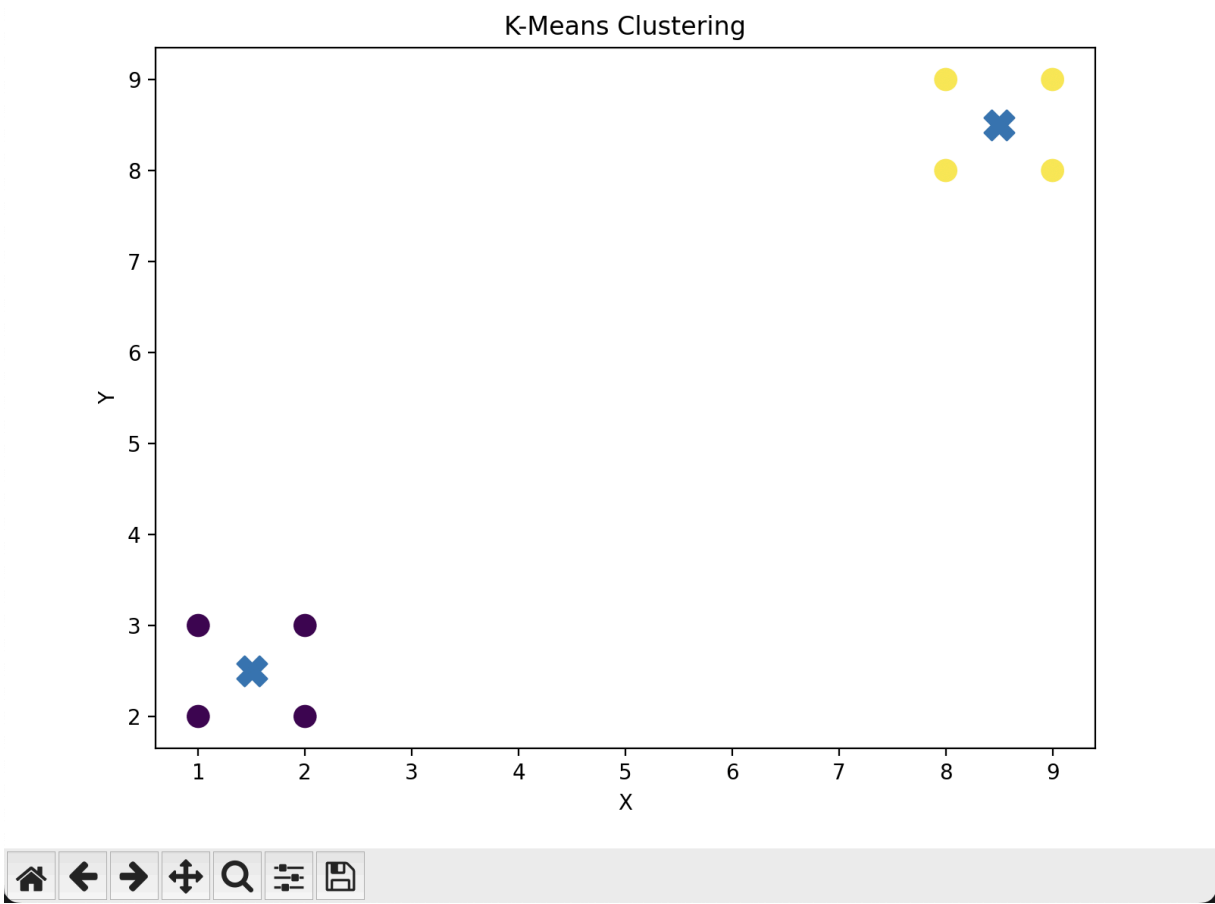
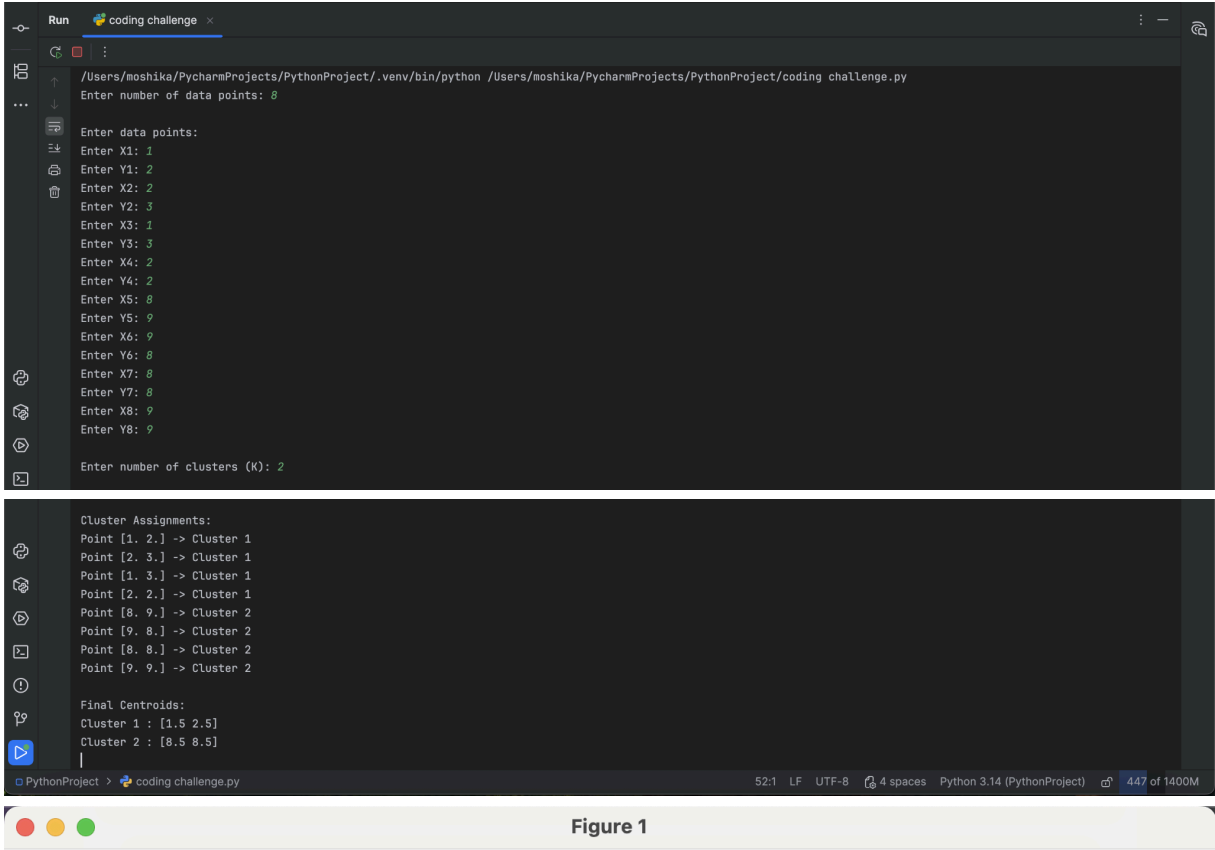
ANSWER:

CODE



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.cluster import KMeans
4 n = int(input("Enter number of data points: "))
5 X = []
6 print("\nEnter data points:")
7 for i in range(n):
8     x = float(input(f"Enter X{i + 1}: "))
9     y = float(input(f"Enter Y{i + 1}: "))
10    X.append([x, y])
11 X = np.array(X)
12 k = int(input("\nEnter number of clusters (K): "))
13 model = KMeans(
14     n_clusters=k,
15     random_state=42,
16     n_init=10
17 )
18 model.fit(X)
19 labels = model.labels_
20 centers = model.cluster_centers_
21 print("\nCluster Assignments:")
22 for i in range(n):
23     print(
24         "Point", X[i],
25         "\n-> Cluster", labels[i] + 1
26     )
27 print("\nFinal Centroids:")
28 for i in range(k):
29     print(
30         "Cluster", i + 1,
31         "\nCentroids", centers[i]
32     )
33 plt.figure(figsize=(8, 6))
34 plt.scatter(
35     X[:, 0],
36     X[:, 1],
37     c=labels,
38     s=100
39 )
40 plt.scatter(
41     centers[:, 0],
42     centers[:, 1],
43     marker="X",
44     s=200
45 )
46 plt.xlabel("X")
47 plt.ylabel("Y")
48 plt.title("K-Means Clustering")
49 plt.show()
```

OUTPUT



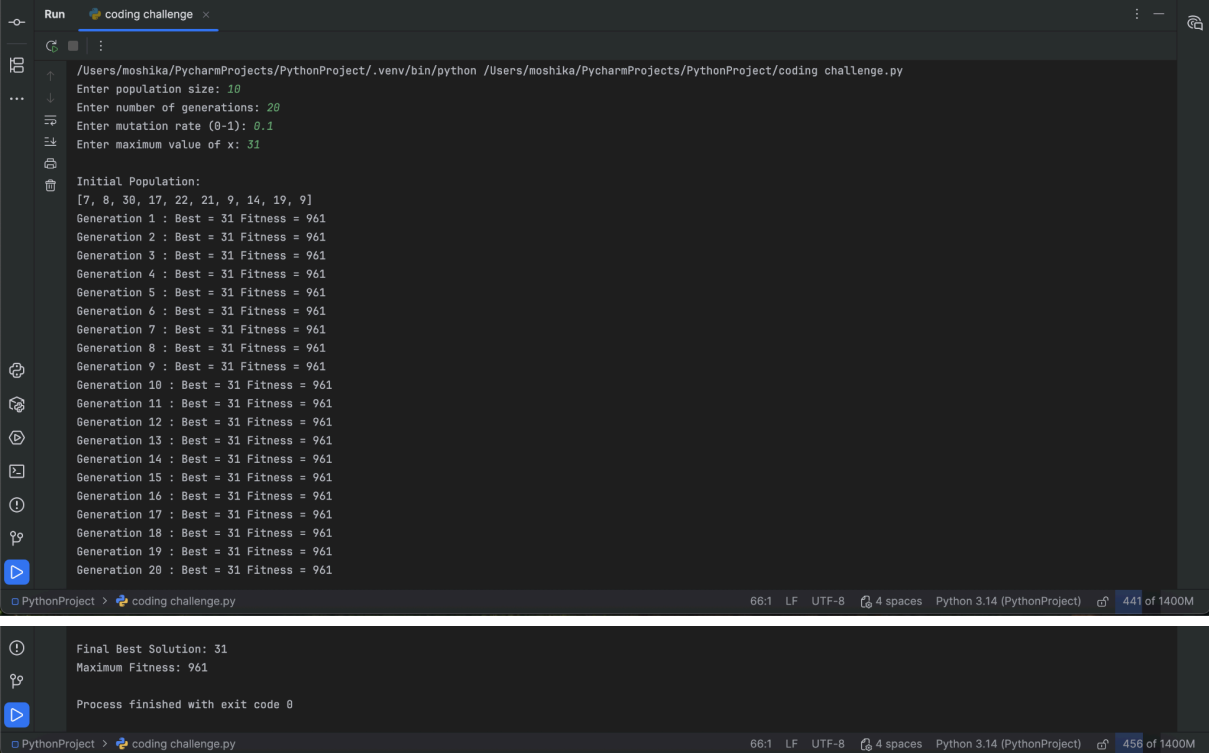
10. Write a Python program to implement a simple Genetic Algorithm to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.

ANSWER:

CODE

```
coding challenge.py
1 import random
2 def fitness(x):
3     return x ** 2
4 population_size = int(
5     input("Enter population size: ")
6 )
7 generations = int(
8     input("Enter number of generations: ")
9 )
10 mutation_rate = float(
11     input("Enter mutation rate (0-1): ")
12 )
13 maximum = int(
14     input("Enter maximum value of x: ")
15 )
16 population = [
17     random.randint(-maximum, maximum)
18     for _ in range(population_size)
19 ]
20 print("\nInitial Population:")
21 print(population)
22 for generation in range(generations):
23     population.sort(
24         key=fitness,
25         reverse=True
26     )
27     parents = population[:2]
28     new_population = parents.copy()
29     while len(new_population) < population_size:
30         parent1 = random.choice(parents)
31         parent2 = random.choice(parents)
32         child = random.choice([
33             parent1,
34             parent2,
35             (parent1 + parent2) // 2
36         ])
37         if random.random() < mutation_rate:
38             mutation = random.randint(
39                 -2, 2
40             )
41             child += mutation
42             child = max(
43                 0,
44                 min(maximum, child)
45             )
46         new_population.append(child)
47     population = new_population
48     best = max(
49         population,
50         key=fitness
51     )
52     print(
53         "Generation",
54         generation + 1,
55         ": Best =", best,
56         "Fitness =", fitness(best)
57     )
58     best = max(
59         population,
60         key=fitness
61     )
62 print("\nFinal Best Solution:", best)
63 print("Maximum Fitness:", fitness(best))
```

OUTPUT



```
Run coding challenge x
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/coding_challenge.py
Enter population size: 10
Enter number of generations: 20
Enter mutation rate (0-1): 0.1
Enter maximum value of x: 31

Initial Population:
[7, 8, 39, 17, 22, 21, 9, 14, 19, 9]
Generation 1 : Best = 31 Fitness = 961
Generation 2 : Best = 31 Fitness = 961
Generation 3 : Best = 31 Fitness = 961
Generation 4 : Best = 31 Fitness = 961
Generation 5 : Best = 31 Fitness = 961
Generation 6 : Best = 31 Fitness = 961
Generation 7 : Best = 31 Fitness = 961
Generation 8 : Best = 31 Fitness = 961
Generation 9 : Best = 31 Fitness = 961
Generation 10 : Best = 31 Fitness = 961
Generation 11 : Best = 31 Fitness = 961
Generation 12 : Best = 31 Fitness = 961
Generation 13 : Best = 31 Fitness = 961
Generation 14 : Best = 31 Fitness = 961
Generation 15 : Best = 31 Fitness = 961
Generation 16 : Best = 31 Fitness = 961
Generation 17 : Best = 31 Fitness = 961
Generation 18 : Best = 31 Fitness = 961
Generation 19 : Best = 31 Fitness = 961
Generation 20 : Best = 31 Fitness = 961

Final Best Solution: 31
Maximum Fitness: 961

Process finished with exit code 0
```

The screenshot shows a PyCharm Run console window for a file named 'coding_challenge.py'. The script prompts for user input: population size (10), number of generations (20), mutation rate (0.1), and maximum value of x (31). It then displays the initial population and the results for each of the 20 generations, showing a constant best fitness of 961 and a best solution of 31. The console also shows the final best solution and maximum fitness, and that the process finished with exit code 0. The status bar at the bottom indicates the file is encoded in UTF-8, uses 4 spaces for indentation, and is running on Python 3.14.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand